SHOPEASE · BACKEND ENGINEERING

Product Service

Project Report

A FastAPI product catalog service with Azure SQL persistence, schema migrations, API validation, and Docker packaging.

Python | FastAPI | Pydantic | Azure SQL | Docker

Report contents

01 · Project overview

02 · Technology and structure

03 · Architecture and request flow

04 · REST API design

05 · Validation and errors

06 · Database and migrations

07 · Docker build and run

08 · Live API evidence

09 · Testing and verification

10 · Setup, security, and limitations

Verification boundary: the container's health route and Swagger page were exercised live. No SQL server hostname was configured, so database connectivity, migrations, and persistent CRUD are not claimed as live-tested.

01 · PROJECT OVERVIEW

Purpose and scope

ShopEase Product Service exposes a small, independent HTTP API for managing catalog products.

Clients send JSON requests to FastAPI routes. Pydantic validates request data, the route delegates to the product service, and a repository performs data access. The production repository is designed for Azure SQL Database; an in-memory repository is retained for isolated API tests.

Core capabilities

- Create a product
- List, category-filter, and paginate
- Read one product by ID
- Replace product fields
- Delete a product

Operational goals

- Typed request and response schemas
- Stable REST status codes
- Containerized Uvicorn service
- Explicit and guarded SQL migrations
- Testable repository/service boundary

Product fields

Field	Purpose	Validation / storage
id	Product identifier	Database-generated integer
name, description	Display text	Up to 100 and 1000 characters
price	Unit price	Positive, 4 decimal places; DECIMAL(19,4)
stock_quantity	Available stock	Integer ≥ 0
category, status	Catalog grouping and state	Status: Active or Inactive
created_at	Creation timestamp	UTC timestamp from database

02 · TECHNOLOGY AND STRUCTURE

Technology stack

Layer	Technology	Role
Runtime	Python 3.11+	Application language
HTTP API	FastAPI + Uvicorn	Routes, ASGI serving, OpenAPI docs
Validation	Pydantic	Request parsing and response shaping
Persistence	Azure SQL + mssql-python	Parameterized SQL access and tables
Container	Docker / Compose	Repeatable image and runtime
Verification	pytest, Swagger UI, Postman	Automated and manual API checks
Source control	Git / GitHub	Versioning and collaboration

Code organization

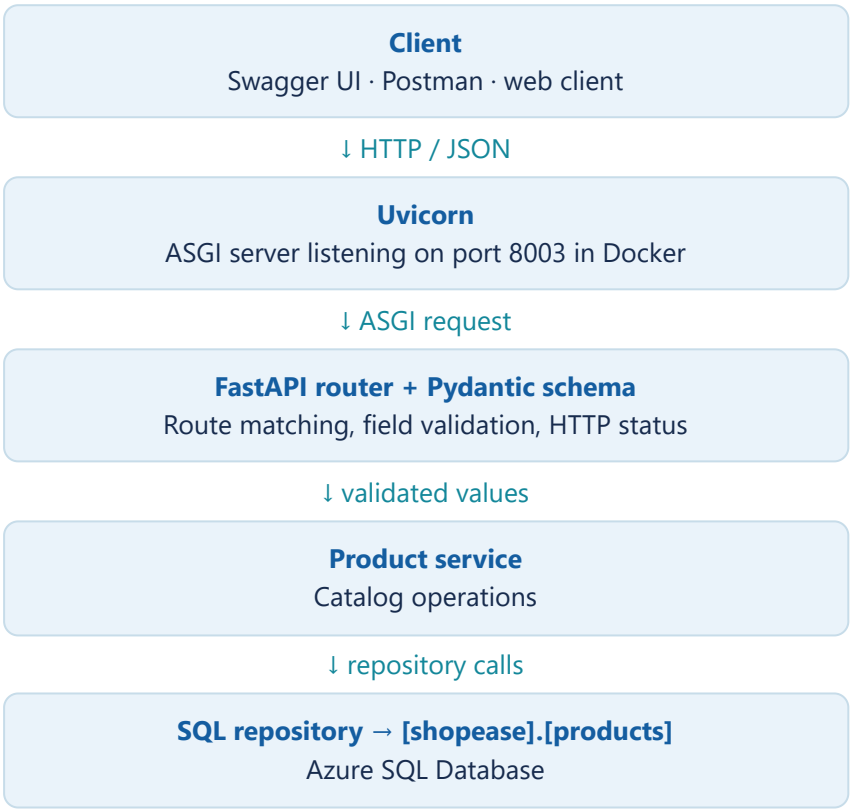
```
app/
  main.py                 FastAPI app and health route
  config/settings.py      environment-backed settings
  routers/                HTTP endpoints
  schemas/                Pydantic validation
  models/                 product data model
  services/               application-level operations
  repositories/           SQL and in-memory data access
  db/                     connection and migration runner
tests/                    API and database-safety checks
Dockerfile / compose.yaml container build and run settings
```

> **Separation of concerns:** API routes do not issue SQL directly. Repository implementations are selected below the service layer, which keeps HTTP behavior independently testable.

03 · ARCHITECTURE AND REQUEST FLOW

Layered request path

Each request passes through a short sequence of typed layers. This lets validation happen before business logic or a database operation is attempted.



Response path

The repository returns a product model, the response schema shapes it, and FastAPI serializes JSON with the route's declared status code. Unexpected exceptions are logged server-side and return a generic 500 response.

Design benefits

- Input validation is consistent and visible in generated OpenAPI docs.
- The service layer provides a stable seam between routes and storage.
- Tests can inject an in-memory service without requiring SQL.
- The SQL repository uses parameterized statements rather than string-built values.

04 · REST API DESIGN

Product endpoints

Method	Path	Behavior	Success
POST	/api/products/	Create product	201
GET	/api/products/	List / filter / paginate	200
GET	/api/products/{id}	Read by ID	200
PUT	/api/products/{id}	Replace product fields	200
DELETE	/api/products/{id}	Delete by ID	204

Create request example

```
{
  "name": "Headphones",
  "description": "Wireless headphones",
  "price": 25.50,
  "stock_quantity": 20,
  "category": "Accessories",
  "status": "Active"
}
```

List and error behavior

`GET /api/products/?category=Accessories&skip=0&limit=10` supports optional case-insensitive category matching and pagination. Unknown product IDs return 404. Invalid path or request data returns 422.

> Try endpoints interactively at `http://localhost:8003/docs` when the service is running. Use the same host and port in Postman.

05 · VALIDATION AND ERRORS

Guarding the API boundary

Pydantic schemas reject malformed or out-of-range input before it reaches the product service. Unexpected fields are forbidden so client mistakes are not silently ignored.

Input	Rule	Example rejected
name / category	Trimmed, non-empty, max 100 chars	Spaces only
description	Optional, max 1000 chars	Whitespace-only text
price	Strict positive number, max 4 decimals	0, negative, 0.00001
stock_quantity	Strict integer ≥ 0	-1 or a decimal
status	Active or Inactive	Pending
product_id	Integer greater than zero	0 or non-numeric text

HTTP status summary

200 OK
Read or update succeeded.

201 Created
Product created.

204 No Content
Delete succeeded.

404 Not Found
Product ID does not exist.

422 Unprocessable Entity
Invalid request, path, or query.

500 Internal Server Error
Unexpected server-side failure.

Database boundary: 422 validation can be verified without a database. Successful product CRUD requires a reachable, migrated SQL database; this report does not imply otherwise.

06 · DATABASE AND MIGRATIONS

Persistence and schema safety

Product records are designed for Azure SQL Database. Application tables and migration state are isolated in the `shopease` schema.

Object	Purpose
[shopease].[products]	Product data, PK, checks, and category index
[shopease].[__schema_migrations]	Applied migration versions and SHA-256 checksums
[shopease].[__migration_lock]	Application-level migration lock record

Migration safeguards

1. Explicit CLI requires `--database` to match `SQL_DATABASE` and refuses `master`.
2. Engine and current database are checked before DDL.
3. Migrations run in order under a database application lock and a transaction.
4. Previously applied files are checksum-verified; edited, missing, or unknown versions fail.
5. Postconditions are checked before recording the migration and again for the latest schema.
6. Failures roll back and propagate; app startup does not apply schema changes.

> **Not live-applied:** no SQL_HOST was present in the local environment when evidence was captured. A real Azure SQL migration and database CRUD still need to be run against the intended database.

Money precision

API price rules align to `DECIMAL(19,4)` so values outside the database range or scale are rejected rather than silently rounded.

07 · DOCKER BUILD AND RUN

Container output

The Docker image uses Python slim, installs the pinned application requirements, copies the app package, and starts Uvicorn on port 8003.

```
docker build -t shopease-product-service .
docker run -d --name shopease-product-container `
  -p 8003:8003 shopease-product-service
docker logs shopease-product-container
```

PowerShell — Docker verification summary

```
PS D:\ShopEase\shopease-product-service> docker build -t shopease-product-service-report .
#6 [1/5] FROM python:3.12-slim
#7 [2/5] WORKDIR /service · CACHED
#8 [3/5] COPY requirements.txt . · CACHED
#9 [4/5] RUN pip install -r requirements.txt · CACHED
#10 [5/5] COPY app ./app
#11 DONE · naming to shopease-product-service-report:latest

PS> docker run -d --name shopease-report-runtime -p 127.0.0.1:18003:8003 shopease-product-service-report
Container started · Uvicorn listening on 0.0.0.0:8003

PS> curl.exe -i http://127.0.0.1:18003/
HTTP/1.1 200 OK · application/json
{"message":"Welcome to the ShopEase Product Service"}

PS> curl.exe http://127.0.0.1:18003/docs
HTTP 200 OK · Swagger UI available

Container smoke check passed; no database host was configured for this check.
```

Formatted summary of the verified Docker build and runtime output. Container root and Swagger smoke checks each returned HTTP 200. The SQL database was not contacted by these checks.

Where to test

- Health: `http://localhost:8003/`
- Swagger UI: `http://localhost:8003/docs`
- OpenAPI: `http://localhost:8003/openapi.json`

08 · LIVE API EVIDENCE

Running service responses

The project image was started as a container and checked over the published local port. These formatted evidence views document the health response and generated API documentation.

ShopEase Product Service — Live API Output

REQUEST

```
GET http://127.0.0.1:18003/
```

HTTP STATUS

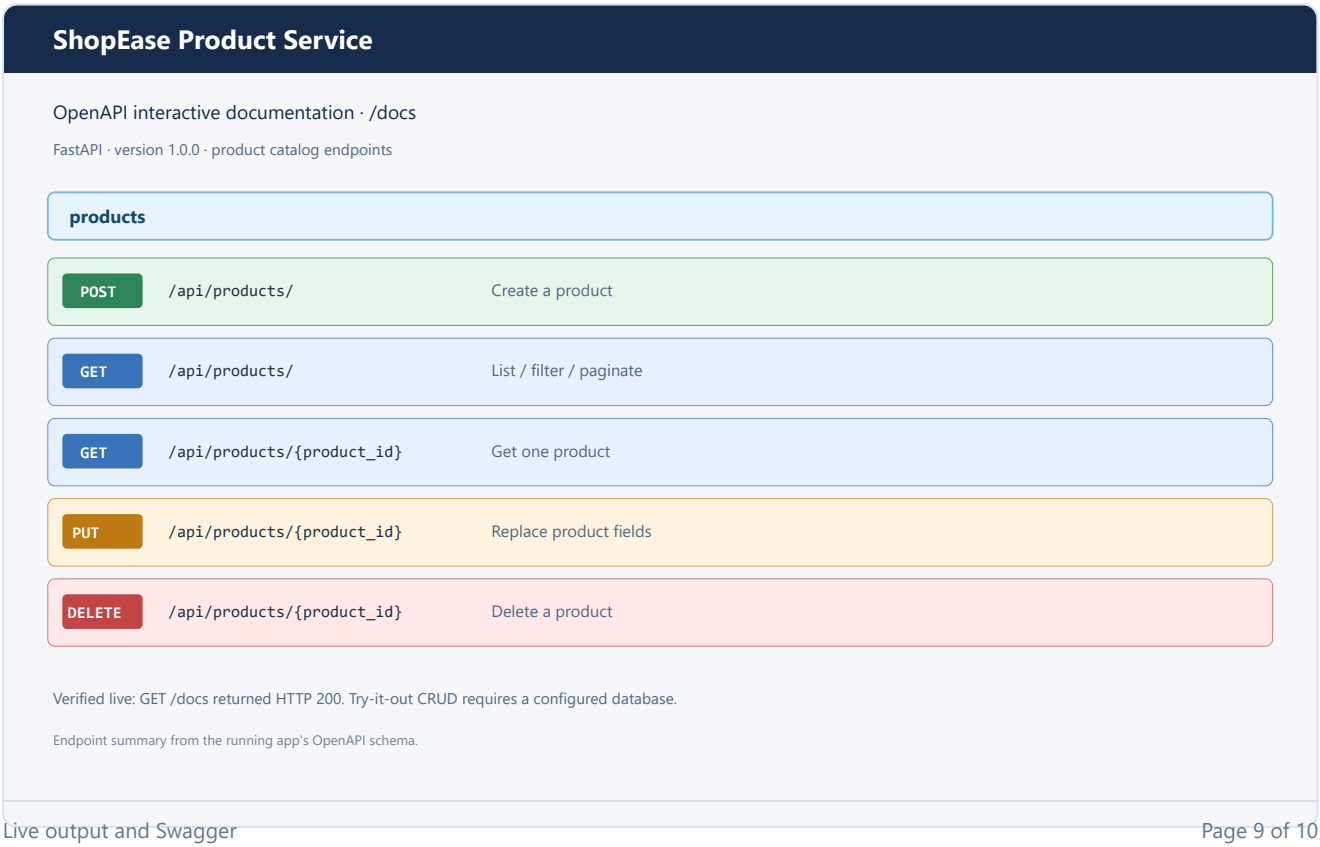
200 OK

RESPONSE BODY · application/json


```
{
  "message" : "Welcome to the ShopEase Product Service"
}
```


Formatted view of the live container response. Health does not test SQL connectivity or product CRUD.

Formatted view of the exact JSON response verified from the live container; GET / returned 200.



Live output and Swagger
GET /docs returned 200 and displayed the API's five product operations.

09 · TESTING AND VERIFICATION

What passed and what remains

Check	Result	What it proves
pytest	26 passed	API behavior and migration-safety unit tests
pip check	Passed	No broken installed requirements
Python compileall	Passed	Application and test modules compile
Docker build	Passed	Image assembles with application dependencies
Container health	HTTP 200	App starts and root route responds
Swagger / OpenAPI	HTTP 200	Docs and schema endpoints are served

Recommended next verification

1. Set the real SQL_HOST and target SQL_DATABASE in local configuration.
2. Run the migration command explicitly against that database.
3. Confirm schema and migration-history postconditions.
4. Exercise POST → GET → PUT → DELETE through Swagger or Postman.
5. Restart the container and verify product data persists.

> **Test boundary:** automated API tests inject an in-memory repository. Migration safety tests use fakes; a real SQL server was not available during this report's verification.